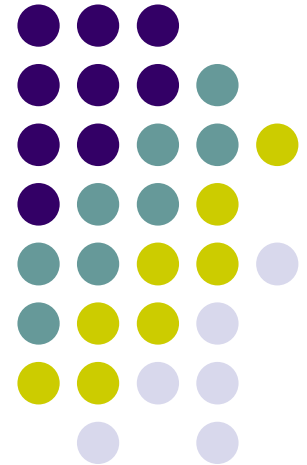


BM201 Görsel Programlamaya Giriş



Chapter 2: C# Programlamaya Giriş

Dr. Öğr. Üyesi Ali Şenol
Bilgisayar Mühendisliği Bölümü
GİBTÜ





Amaç

- Basit C# programı yazabilmek
- Giriş-Çıkış işlemlerini gerçekleştirebilmek
- Temel veri türlerine aşinalık kazanmak
- Aritmetik operatörleri kullanabilmek
- Aritmetik operatörlerin öncelik sırasını öğrenmek
- Karşılaştırma ve eşitlik operatörlerini öğrenmek



C# (Si-Şarp) nedir?

- Microsoft tarafından geliştirilmiş bir dildir.
- C# nesne tabanlı programlamayı destekleyen, basit, kullanışlı ve C ile C++ dillerinden türetilen bir dildir.
- Programlama mantığı ve temeli Java diline çok benzer.
- Öne çıkan özellikleri:
 - Windows ve web programlama için görsel tasarım sağlar.
 - Derlenen bir programlama dilidir.



C# Nitelikleri

- Basit
- Esnek
- Nesne tabanlı programlamanın tüm özelliklerini taşır.
- Otomatik çöp toplayıcı (Garbage collector)



C#'ın Başlıca Uygulamaları

- *Masaüstü Uygulamaları*: Görsel veya ihtiyaç duyulan diğer masaüstü yazılımları geliştirme
- *Class Library*: Başka uygulamalarda kullanılacak sınıf kütüphaneleri yaratma
- *Console Applications*: Text tabanlı uygulamalar (Siyah ekran uygulamaları)
- *Asp.Net Web Applications*: Görsel web uyg.
- *Asp.Net Web Services*: XML web uygulamaları
- *Asp.Net Mobile Web Applications*: Cep telefonları gibi taşınabilir cihazlar için uygulama geliştirme



Çalıştırılabilir Kod Oluşturma

- Yazılan programların çalışması için makine diline dönüştürülmesi gerekir.
- Windows İS'de makine dili dosyaları «exe» uzantılıdır.
- Ör: MerhabaDunya.cs → MerhabaDunya.exe

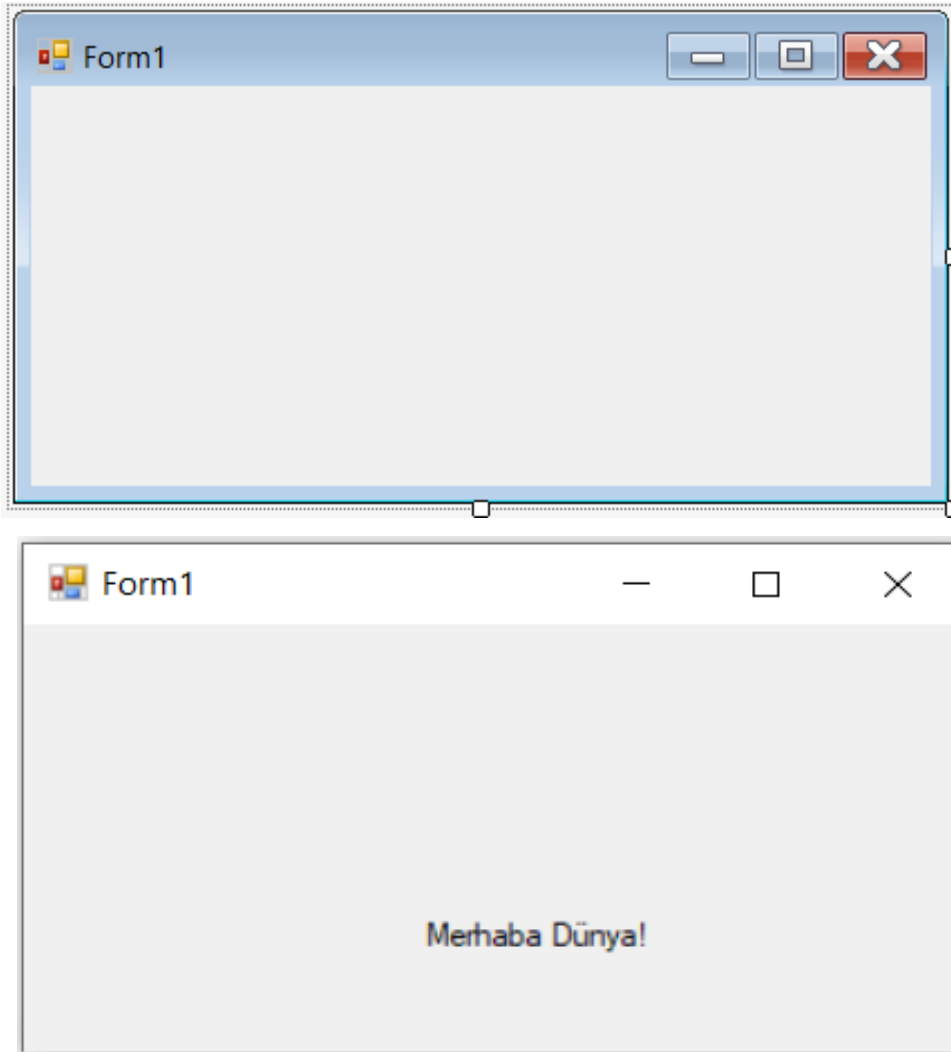
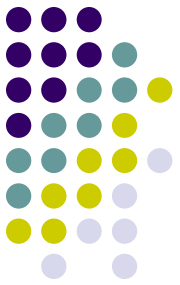
«Merhaba Dünya» Console Uygulaması



```
Program.cs  X
C# MerhabaDunya  MerhabaDunya.Program  Main(string[] args)
1  //Tek satırlık yorum yazmak için "/" kullanılır
2  /*
3   Çok satırlı yorum yazmak için "/*" bloğu arasına yazılır
4   */
5
6
7  using System; // Sistem kütüphanesinin kullanılacağını belirttik ve kodumuza ekledik
8
9  namespace MerhabaDunya //Projemizin adı
10 {
11     0 başvuru
12     class Program //Program adında bir sınıf oluşturduk
13     {
14         0 başvuru
15         static void Main(string[] args) //C dersinde gördüğümüz main fonksiyonunun karşılığı
16         {
17             Console.WriteLine("Merhaba Dünya!"); //Console (Siyah ekrana) ekranına yazma kodu
18             Console.ReadKey(); // Siyah ekranın kapanmaması için kullanıcıdan bir tuşa
19                                     //basmasını istiyoruz C'deki getch'ın karşılığı
20         }
21     }
22 }
```

```
E:\Verilen Dersler\BM201\Örnek Kodlar\MerhabaDunya\MerhabaDunya\bi...
Merhaba Dünya!
```

«Merhaba Dünya» Form Uygulaması



```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
namespace MerhanaDunyaForm
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }
        private void Form1_Load(object sender,
EventArgs e)
        {
            label1.Text = "Merhaba Dünya!";
        }
    }
}
```




C# Veri Türleri

Variable (değişken) tanımlama

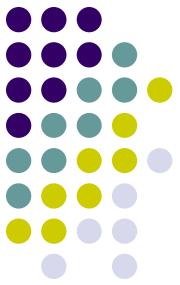
- Değişken tanımlama, hafızada belirtilen veri türü için bir kayıt alanı oluşturmaktır. Bu şekilde kullanıcı verileri kaydedilip ihtiyaç duyulduğunda kullanılabilir
- `<data type> <variable name> = <value>;`
 - `int a=25, b; double d=10.5; gibi`



C# Veri Türleri

Variable (değişken) tanımlamada dikkat edilmesi gereken kurallar

- Değişken adı unique (benzersiz) olmalı
- Değişken isimleri sadece harf rakam ve _ (altçizgi) içerebilir.
- Değişken isimleri mutlaka harf ile başlamalı
- Değişken isimleri büyük küçük harf duyarlıdır (num ve Num farklı iki değişken)
- Değişken isimleri rezerv kelimeleri int, class, bool vs. gibi içeremez. Eğer kullanılacaksa önüne @ konulmalı. Ör: @int gibi
- Bir tür diğer türün içine atanamaz



C# Veri Türleri

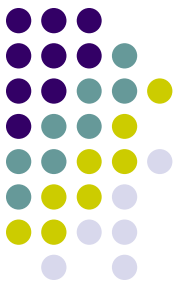
Variable tanımlamlarken dikkat;

- Bir tür diğer türün içine atanamaz.
 - `int num="ali";` veya `string s=25;` gibi
- Bir değişken kullanılmadan önce tanımlanmış olmalı
 - `int num; num=100;` ve `int num=100;`
- Bir değişkenine değeri tanımlandıktan sonra istenen zaman değiştirilebilir.
 - `int num=100; num=200;` gibi
- Tek satırda birden çok tanımlama yapılabilir
 - `int i, j, sayi1;`
- Bir değişkenin içeriği aynı türden başka değişkene kopyalanabilir.
 - `int i=100; int j=i;`



C# Veri Türleri

- Boolean: mantıksal evet-hayır içerebilir
 - `bool content=true; bool noCon=false;`
- Sayısal veri türleri
 - `int → int i=35;`
 - `Long → long y=656454;`
 - `Float → float x; veya float x=12.34f;` (virgüllü olacaksa sonuna f ekliyoruz.)
 - `Double → double y;`
- String (Söz dizisi): `string s="ali şenol";`
- Arrays (Diziler): `int dizi={5,10,15,20};`



C# Veri Türleri

Data Types	Memory Size	Range
char	1 byte	-128 to 127
signed char	1 byte	-128 to 127
unsigned char	1 byte	0 to 255
short	2 byte	-32,768 to 32,767
signed Short	2 byte	-32,768 to 32,767
unsigned Short	2 byte	0 to 65,535
int	4 byte	-2,147,483,648 to 2,147,483,647
signed int	4 byte	-2,147,483,648 to 2,147,483,647
unsigned int	4 byte	0 to 4,294,967,295
long	8 byte	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
signed long	8 byte	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
unsigned long	8 byte	0 to 18,446,744,073,709,551,615
float	4 byte	$1.5 * 10^{-45}$ to $3.4 * 10^{38}$ (7 Digit)
double	8 byte	$5 * 10^{-324}$ to $1.7 * 10^{308}$
decimal	16 byte	$-7.9 * 10^{28}$ to $7.9 * 10^{28}$



C# Operatörleri

Operatör Nedir?

Operatörler önceden tanımlanmış birtakım matematiksel ya da mantıksal işlemleri yapmak için kullanılan özel karakterler ya da karakterler topluluğudur.

Operatörlerin işlem yapabilmek için ihtiyaç duydukları değerlere ise “operand” denir.

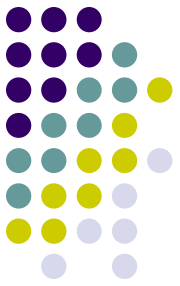
ÖR: $x+y \rightarrow x$ ve y operand, $+$ ise operatördür.

C# Operatörler Çeşitleri (Görevlerine Göre)



- Aritmetik Operatörler
- Karşılaştırma Operatörleri
- Mantıksal Operatörler
- Bitsel Operatörler
- Atama ve İşlemlili Atama Operatörleri
- Özel Amaçlı Operatörler

C# Operatörler Çeşitleri (Görevlerine Göre)



Aritmetik Operatörler

`+` , `-` , `*` , `/` , `%` , `++` , `--`

Karşılaştırma Operatörleri

`>` , `<` , `<=` , `>=` , `==` , `!=` , *as* , *is*

Mantıksal Operatörler

`||` , `&&` , `!`

Bitsel Operatörler

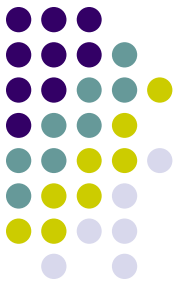
`|` , `&` , `~` , `^` , `<<` , `>>`

Atama ve İşlemlili Atama Operatörleri

`=` , `*=` , `/=` , `%=` , `+=` , `-=` , `<<=` , `>>=` , `&&=` , `^=` , `|=`

Özel Amaçlı Operatörler

`?` , `:` , `()` , `[]` , `+` , `-` , `&` , `*` , `->` , `..` , *new* , *checked* , *unchecked* , *typeof* , *sizeof*



Aritmetik Operatörler

□ **Aritmetik Operatörler: +, -, *, /**

Matematikteki toplama, çıkarma, çarpma ve bölme operatörleridir. Bölme işleminde kullanılan değerın tipine göre farklı sonuçlar elde edilebilir.

- `int i = 20 + 30;`
- `float f = 12.34f + 56.78f;`

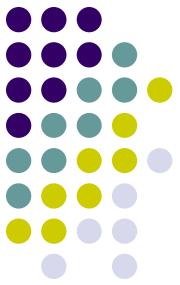


Aritmetik Operatörler

□ % Operatörü:

Bölümden sonra kalanı bulmak yani mod almak amacıyla kullanılır. Tüm sayısal türler için kullanılabilir.

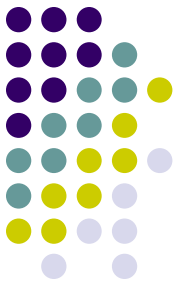
- $x = 10 \% 3$ $x = 1$
- $y = -7 \% 4$ $y = -3$
- $f = 2.5f \% 1.2f$ $f = 0.1$



Aritmetik Operatörler

□ ++, -- Operatörleri:

Önüne ya da sonuna geldiği değişkenin değerini 1 artırır ya da 1 azaltır. İki şekilde kullanılırlar: Önek (prefix) ve Sonek (postfix). Bütün sayısal değerlerde kullanılabilir.



Aritmetik Operatörler

□ ++, -- Operatörleri:

Arttırma ve Azaltma operatörlerinde dikkat edilmesi gereken durumlar şu şekildedir:

a = 4

b = a++

a=5, b=4

a = 4

b = ++a

a=5, b=5

a = 4

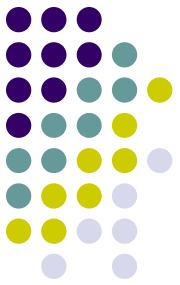
b = a--

a=3, b=4

a = 4

b = --a

a=3, b=3



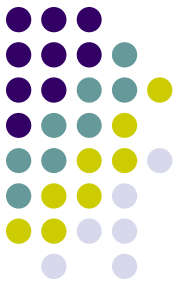
Aritmetik Operatörler

❏ ++, -- Operatörleri Örnek Kodu:

```
using System;

namespace PrefixPostfix2
{
    class Program
    {
        static void Main(string[] args)
        {
            int a = 5; int b = 10;
            int c = 20; int d = 50;
            float f = 3.4f;

            Console.WriteLine("a="+ ++a + "\nb="+ b++ + "\nc="+ c-- + "\nd="+ --d + "\nf="+ f++);
            Console.ReadKey();
        }
    }
}
```



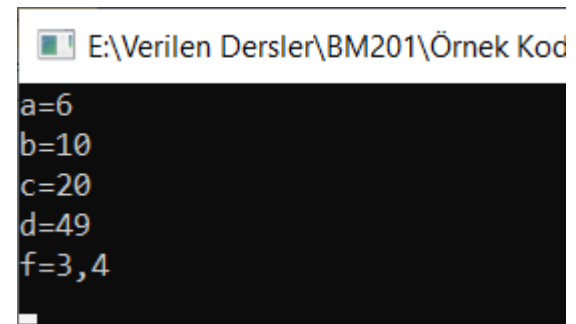
Aritmetik Operatörler

❏ ++, -- Operatörleri Örnek Kodu:

```
using System;

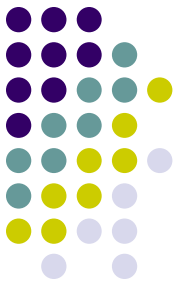
namespace PrefixPostfix2
{
    class Program
    {
        static void Main(string[] args)
        {
            int a = 5; int b = 10;
            int c = 20; int d = 50;
            float f = 3.4f;

            Console.WriteLine("a="+ ++a + "\nb="+ b++ + "\nc="+ c-- + "\nd="+ --d + "\nf="+ f++);
            Console.ReadKey();
        }
    }
}
```



E:\Verilen Dersler\BM201\Örnek Kod

```
a=6
b=10
c=20
d=49
f=3,4
```



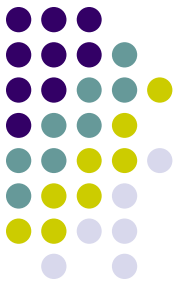
Aritmetik Operatörler

❏ ++, -- Operatörleri Örnek Kodu:

```
using System;

namespace PrefixPostfix2
{
    class Program
    {
        static void Main(string[] args)
        {
            int a = 10; int b, c;
            b = a++;
            c = ++a;
            Console.WriteLine("a {0}", a);
            Console.WriteLine("b {0}", b);
            Console.WriteLine("c {0}", c);

            Console.ReadKey();
        }
    }
}
```



Aritmetik Operatörler

□ ++, -- Operatörleri Örnek Kodu:

```
using System;
```

```
namespace PrefixPostfix2
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            int a = 10; int b, c;
```

```
            b = a++;
```

```
            c = ++a;
```

```
            Console.WriteLine("a {0}", a);
```

```
            Console.WriteLine("b {0}", b);
```

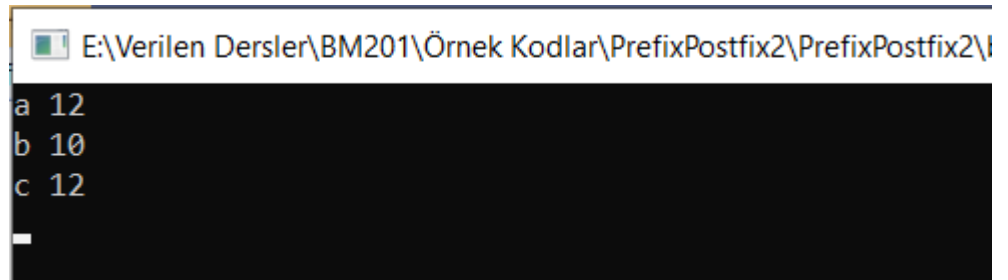
```
            Console.WriteLine("c {0}", c);
```

```
            Console.ReadKey();
```

```
        }
```

```
    }
```

```
}
```



```
E:\Verilen Dersler\BM201\Örnek Kodlar\PrefixPostfix2\PrefixPostfix2\l  
a 12  
b 10  
c 12  
_
```



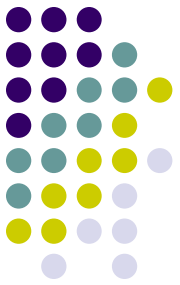

Karşılaştırma Operatörler

> ve < Operatörleri:

İki sayısal değerin birbirlerine göre büyüklüğünü ve küçüklüğünü kontrol eder. Bu operatörlerin ürettiği değerler “true” veya “false” değerleridir. < operatörü için eğer birinci operand ikinci operanddan küçük ise “true” aksi durumda “false” üretir. > operatörü için ise birinci operand ikinciden küçükse “false” büyükse “true” değerlerini üretir.

<= ve >= Operatörleri:

> ve < operatörleriyle tamamen aynıdır. Yalnızca eşitlik söz konusu olduğunda da “true” değeri üretirler.



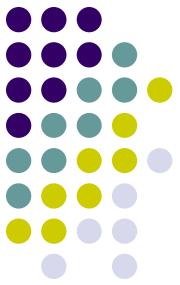
Karşılaştırma Operatörler

❏ **<, <= ve >= Operatörleri Örnek Kodu:**

```
using System;

namespace OperatorsRelativity
{
    class Program
    {
        static void Main(string[] args)
        {
            bool b1 = 20 < 30;
            bool b2 = 30 < 20;
            bool b3 = 20.53 > 20.532;
            bool b4 = 12.3f > 11.9f; bool b5 = 15 <= 15;
            bool b6 = 20 >= 30;

            Console.WriteLine(b1); Console.WriteLine(b2); Console.WriteLine(b3);
            Console.WriteLine(b4); Console.WriteLine(b5); Console.WriteLine(b6);
            Console.ReadKey();
        }
    }
}
```



Karşılaştırma Operatörler

❏ <, <=, > ve >= Operatörleri Örnek Kodu:

```
using System;
```

```
namespace OperatorsRelativity
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            bool b1 = 20 < 30;
```

```
            bool b2 = 30 < 20;
```

```
            bool b3 = 20.53 > 20.532;
```

```
            bool b4 = 12.3f > 11.9f; bool b5 = 15 <= 15;
```

```
            bool b6 = 20 >= 30;
```

```
            Console.WriteLine(b1); Console.WriteLine(b2); Console.WriteLine(b3);
```

```
            Console.WriteLine(b4); Console.WriteLine(b5); Console.WriteLine(b6);
```

```
            Console.ReadKey();
```

```
        }
```

```
    }
```

```
}
```

```
E:\Verilen Dersler\BM201\Örnek Kod
True
False
False
True
True
False
```

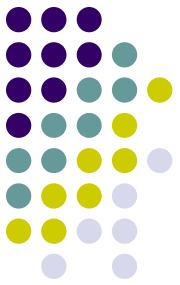


Karşılaştırma Operatörler

== ve != Operatörleri:

İki değerin birbirine eşitlik durumunu kontrol eder. == operatörü iki değer birbirine eşitse, != ise birbirine eşit değilse true sonucunu döndürür.

- `bool b1 = 100 == 100;` **true**
- `bool b2 = 100 != 100;` **false**
- `bool b3 = 100 == 99;` **false**
- `bool b3 = 100 != 99;` **true**



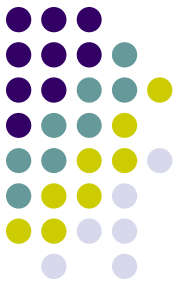
Mantıksal Operatörler

&& (VE) Operatörü:

“true” ya da “false” değerindeki operandları mantıksal VE ile işler. Operandlardan biri “false” ise “false” değeri üretir. Kullanımı: `<ifade> && <ifade>`

Doğruluk Tablosu

1.Operand	2.Operand	Sonuç
False	False	False
False	True	False
True	False	False
True	True	True



Mantıksal Operatörler

❑ ÖR && (VE) Operatörü

- `bool b1 = 25 < 15 && 5 == 50 → False`
- `bool b1 = 15 < 25 && 5 != 50 → True`
- `bool b3 = -12.35f > -12.34f && 0 != 1 → False`



Mantıksal Operatörler

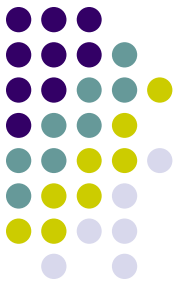
(VEYA) Operatörü:

“true” ya da “false” değerindeki operandları mantıksal VE ile işler. Operandlardan en az biri “true” ise sonuç “ture” olur.

Kullanımı: <ifade> **||** <ifade>

Doğruluk Tablosu

1.Operand	2.Operand	Sonuç
False	False	False
False	True	True
True	False	True
True	True	True



Mantıksal Operatörler

❑ ÖR || (VEYA) Operatörü

- `bool b1 = 25 < 15 || 5 == 50 → False`
- `bool b1 = 25 < 15 || 5 != 50 → True`
- `bool b3 = -12.35f > -12.34f || 0 != 1 → True`



Bitisel Operatörler

□! (DEĞİL) Operatörü:

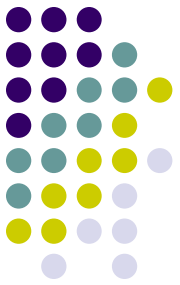
Tek operand alır ve mantıksal değil işlemini gerçekleştirir.

Kullanımı: **!** <ifade>

Doğruluk Tablosu

Operand	Sonuç
False	True
True	False

- `bool b1 = ! (25 < 10) ;`
 - **true**
- `bool b2 = ! (10 != 20) ;`
 - **false**



Bitisel Operatörler

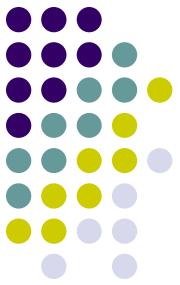
□ ~ (Bitisel DEĞİL) Operatörü:

Tek operand alır ve Değişkendeki tüm bitlerin değilini alır.

Kullanımı: ~ <ifade>

- 0000 1111 sayısının bitisel değili 1111 0000 olur.

```
byte b1 = 254; //11111110
byte b2 = (byte)~b1; //00000001
Console.WriteLine(b2);
```



Bitsel Operatörler

□ & (Bitsel VE) Operatörü:

Operandların tüm bitlerini karşılıklı VE işlemine tabi tutar

byte a = 153;

byte b = 104;

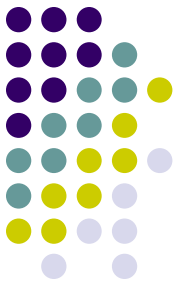
byte x = (byte)(a & b)

x = 8

153 : 1001 1001

104 : 0110 1000

VE 0000 1000



Bitisel Operatörler

□ | (Bitisel VEYA) Operatörü:

Operandların tüm bitlerini karşılıklı VEYA işlemine tabi tutar.

byte a = 153;

byte b = 104;

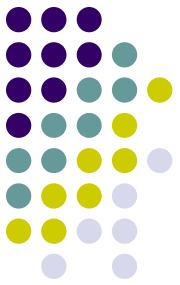
byte x = (byte)(a | b)

x = 249

153 : 1001 1001

104 : 0110 1000

VEYA 1111 1001



Bitisel Operatörler

□ ^ (Bitisel Özel VEYA) Operatörü:

Operandların tüm bitlerini karşılıklı XOR işlemine tabi tutar.

byte a = 153;

byte b = 104;

byte x = (byte)(a ^ b)

x = 241

153 : 1001 1001

104 : 0110 1000

XOR 1111 0001



Bitisel Operatörler

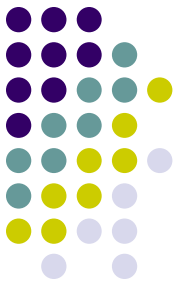
□ << (Bitisel Sola Kaydırma) Operatörü:

Verilen bir tamsayının bitlerinin istenilen sayıda sola doğru ötelenmesini sağlar.

ifade << öteleme sayısı

byte b = 0xFF //1111 1111

byte c = (byte)(b << 4) //1111 0000



Bitisel Operatörler

> (Bitisel Sağa Kaydırma) Operatörü:

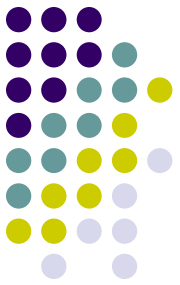
Verilen bir tamsayının bitlerinin istenilen sayıda sağa doğru ötelenmesini sağlar.

ifade >> öteleme sayısı

```
byte b = 0xFF           //1111 1111
```

```
byte c = (byte)(b >> 4) //0000 1111
```

Atama ve İşlemler Atama Operatörleri



□ = (Atama) Operatörü:

Bir değişkene herhangi bir değer atamak için kullanılır.

<değişken> = <atanacak değer>

```
int a=5;
```

```
string ad="ali";
```




Atama ve İşlemlili Atama Operatörler

□ $\ast=$, $/=$, $\% =$, $+=$, $-=$, $<<=$, $>>=$,
 $\&=$, $\wedge=$, $|=$

Bu operatörler operatörler işlemlili atama operatörleri olarak geçer ve işlem sonucunun işlenen operanda atandığı durumlarda kullanılır.

- $x = x + y$; ifadesi yerine $x += y$;
- $a = a * b$; ifadesi yerine $a *= b$;
- $b = b << n$; ifadesi yerine $b <<= n$;



C# Operatör Önceliği

- Bazı durumlarda birden fazla operatör kullanılmış olabilir. Bu tür durumlarda önceden belirlenmiş olan sıraya göre işlem yapılır.
- ÖR: $y = 5 + 10 * 6$
 - *Önce toplama mı yoksa çarpma mı yapılacak?*



Operatör Öncelik Tablosu

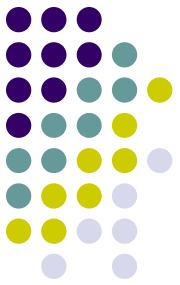
Birinci Öncelikliler	x.y (nesne erişim operatörü) , f(x) , dizi[x], x++ , x-- , new , typeof , checked , unchecked	Soldan sağa
Unary Operatörler	+ , - , ! , ~ , ++x , --x , (tür)x	Sağdan sola
Çarpma Bölme	* , / , %	Soldan sağa
Toplama ve Çıkarma	+ , -	Soldan sağa
Kaydırma Operatörleri	<< , >>	Soldan sağa
İlişkisel ve Tür Testi	< , > , <= , >=	Soldan sağa
Eşitlik Operatörleri	== , !=	Soldan sağa
Mantıksal VE (AND)	&	Soldan sağa
Mantıksal Özel Veya (XOR)	^	Soldan sağa
Mantıksal VEYA (OR)		Soldan sağa
Koşul VE	&&	Soldan sağa
Koşul VEYA		Soldan sağa
Koşul Operatörü	? :	Sağdan sola
Atama ve İşlemlili Atama	= , *= , /= , %= , += , -= , <= , >= , &= , ^= , =	Sağdan sola



İşlem Önceliği

ÖR: $y = 2 * 5 * 5 + 15 / 5 - 7$ işleminin sonucu nedir?

$$\begin{aligned}\text{Çözüm: } y &= (2 * 5) * 5 + 15 / 5 - 7 \\ &= (10 * 5) + (15 / 5) - 7 \\ &= (50 + 3) - 7 \\ &= 53 + 7 \\ &= 46\end{aligned}$$

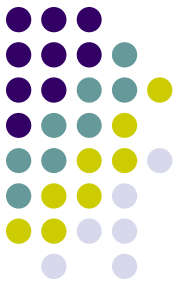


C# Aritmetik Operatörler

ÖR Kod:

```
using System;

namespace OperatorOncelikSirasi
{
    class Program
    {
        static void Main(string[] args)
        {
            int i = 5 * 4 / 6;
            Console.WriteLine(i);
            i = 5 * (4 / 6);
            Console.WriteLine(i);
            Console.ReadKey();
        }
    }
}
```



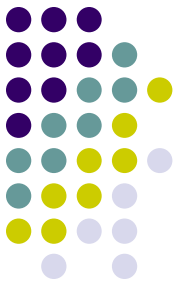
C# Aritmetik Operatörler

ÖR Kod:

```
using System;

namespace OperatorOncelikSirasi
{
    class Program
    {
        static void Main(string[] args)
        {
            int i = 5 * 4 / 6;
            Console.WriteLine(i);
            i = 5 * (4 / 6);
            Console.WriteLine(i);
            Console.ReadKey();
        }
    }
}
```

```
E:\Verilen Dersler\BM201\Örnek Kodlar\OperatorOncelikSirasi\OperatorC
3
0
_
```



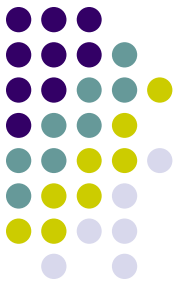
C# Aritmetik Operatörler

ÖR Kod:

```
using System;
```

```
namespace OperatorOncelikSirasi
{
    class Program
    {
        static void Main(string[] args)
        {
            int i = 4 + -6;
            Console.WriteLine(i);

            Console.ReadKey();
        }
    }
}
```



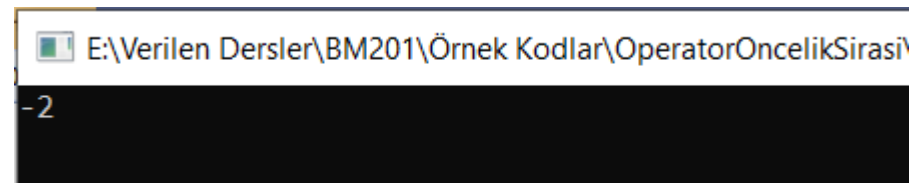
C# Aritmetik Operatörler

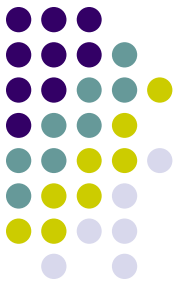
ÖR Kod:

```
using System;
```

```
namespace OperatorOncelikSirasi
{
    class Program
    {
        static void Main(string[] args)
        {
            int i = 4 + -6;
            Console.WriteLine(i);

            Console.ReadKey();
        }
    }
}
```



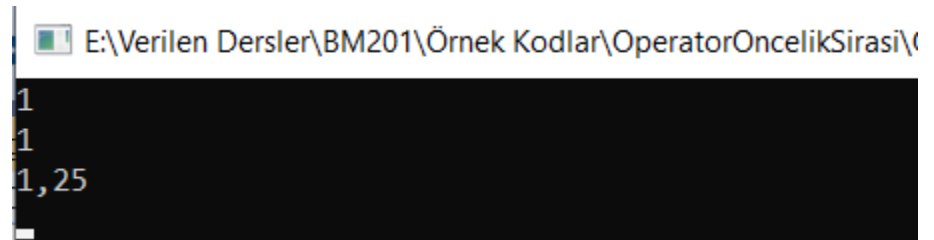


C# Aritmetik Operatörler

ÖR Kod:

```
using System;
namespace OperatorOncelikSirasi
{
    class Program
    {
        static void Main(string[] args)
        {
            int i = 5 / 4;
            float f1 = 5 / 4;
            float f2 = 5f / 4f;

            Console.WriteLine(i);
            Console.WriteLine(f1);
            Console.WriteLine(f2);
            Console.ReadKey();
        }
    }
}
```



E:\Verilen Dersler\BM201\Örnek Kodlar\OperatorOncelikSirasi\

```
1
1
1,25
```



Özel Amaçlı Operatörler

□ . Operatörü

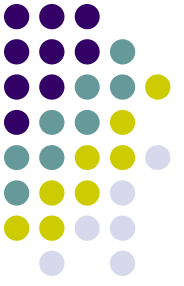
- Bir sınıfın ya da yapının elemanlarına ulaşmak için kullanılır.
 - `Console.WriteLine();`

□ New Operatörü

- Yeni nesne tanımlamak için kullanılır.
 - `int[] arr= new int[10];`

□ checked ve unchecked Operatörleri

- Veri türünü daha küçük tiplere dönüştürürken veri kaybı olduğunda hata vermesini istiyorsak ***checked***; istemiyorsak ***unchecked*** kullanılır.



TEŞEKKÜRLER